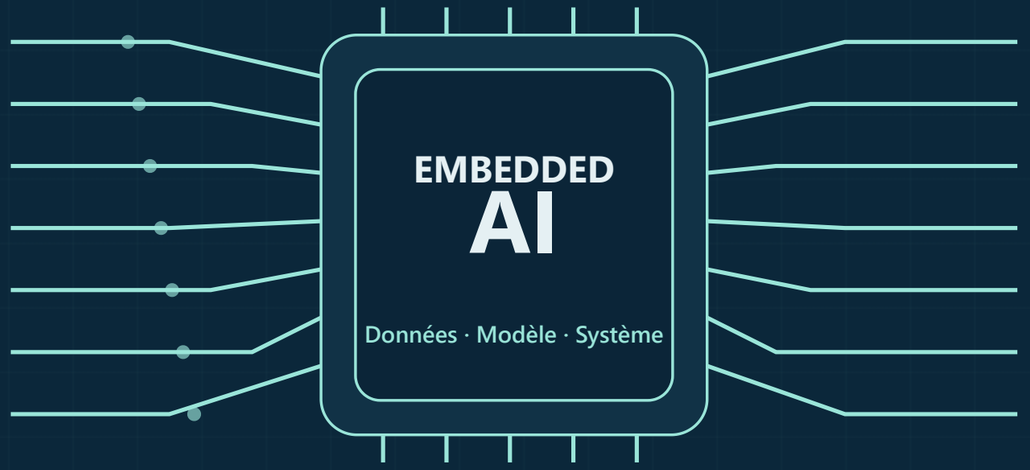
MODÈLES · CALCUL · MÉMOIRE · PREUVES

Une IA utile. Un appareil qui tient ses promesses.

IA pour les systèmes embarqués · Du besoin à la preuve · 3 heures

Roland Cédric TAYO



Le service réel fixe le budget.

Cinq questions.
Un parcours à construire.

Ce cours ouvre le module de 14 h de cours et 14 h de TD ; les quiz sont inclus dans les 3 heures.

- **Pour qui ?** Définir un service utile.
- **Comment apprend-elle ?** Suivre un petit réseau complet.
- **Comment l'embarquer ?** Déployer et compter ses coûts.
- **Comment l'alléger ?** Tester deux leviers concrets.
- **Quelle preuve ?** Mesurer, diagnostiquer et décider.

01 **Pour qui ?**
Besoin et histoire · 20 min

02 **Comment apprend-elle ?**
Un réseau complet · 35 min

03 **Comment l'embarquer ?**
Déploiement et coûts · 30 min

04 **Comment l'alléger ?**
Deux leviers · 35 min

05 **Quelle preuve ?**
Enquête + décision · 50 min

Quiz : 6 · 13 · 19 · 26 · 31 · pause : 10 min

Une personne veut agir sans utiliser ses mains.

Exemple : allumer une lampe en disant « lumière », même sans Internet.

> **Utilité :** une commande accessible quand les mains sont occupées.

> **Difficulté :** les voix et le bruit varient ; le son n'est jamais identique.

> **Choix :** un interrupteur suffit parfois ; la voix doit apporter une aide réelle.



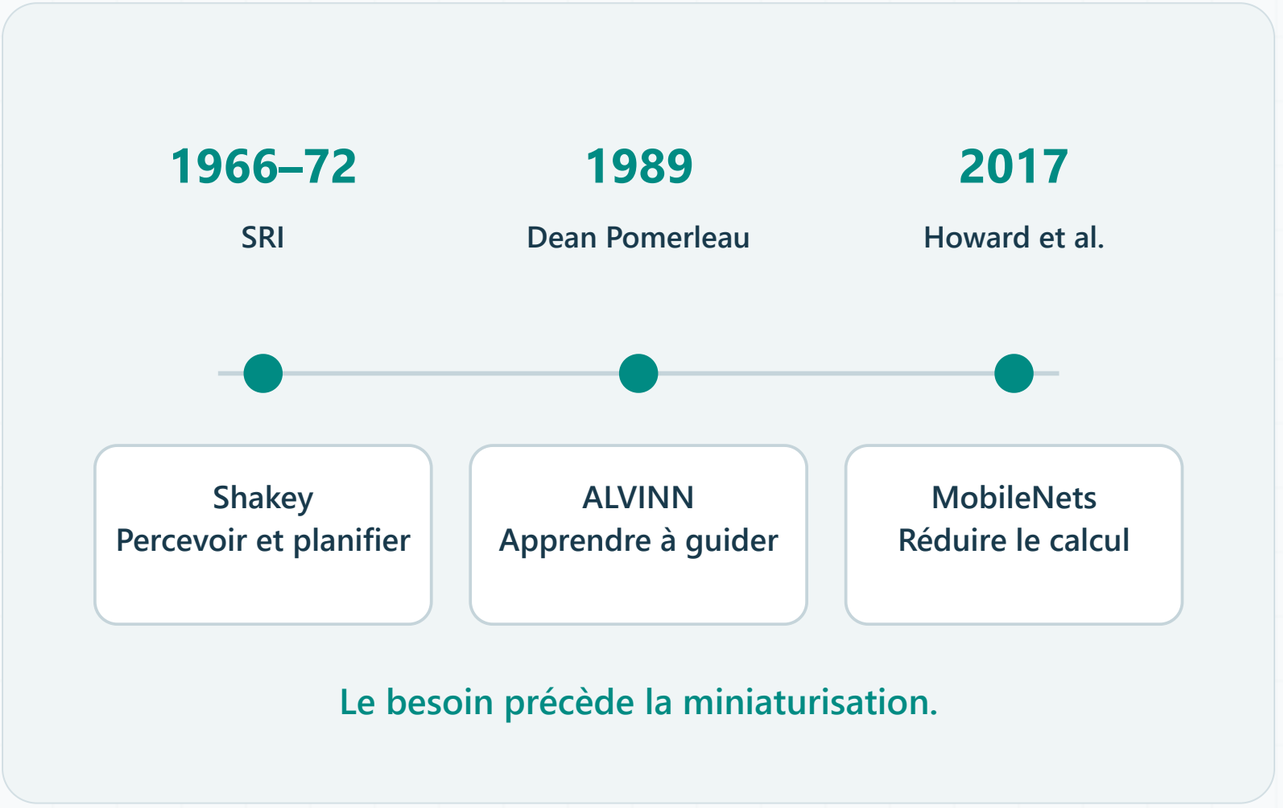
L’histoire : un obstacle, une réponse, une limite.

Le besoin de percevoir et d’agir précède la possibilité de calculer dans une petite puce.

> **SRI · Shakey, 1966–1972 :** percevoir et planifier ; le calcul reste déporté.

> **Pomerleau · ALVINN, 1989 :** apprendre à suivre une route, sous conditions.

> **Howard et al. · MobileNets, 2017 :** réduire le coût de la vision mobile.



Dans l’objet, à proximité ou sur un serveur ?

Le lieu du calcul est un choix d’architecture, pas une obligation de tout rendre local.

Dans l’objet : l’inférence fonctionne à bord ; sa mémoire et son énergie sont limitées.

À proximité : une passerelle traite les données ; la liaison locale reste nécessaire.

À distance : davantage de ressources, mais transmission et dépendance réseau.



Où reconnaître la commande ?

Deux exigences : fonctionner sans Internet et garder tout l'audio dans l'objet. Quelle architecture les respecte ?

?

Votre raisonnement avant la réponse.

Vérifiez chaque exigence séparément avant de choisir.

- **A** L'objet prépare le son et reconnaît la commande.
- **B** L'objet prépare le son ; une passerelle reconnaît la commande.
- **C** L'objet prépare le son ; un serveur reconnaît la commande.

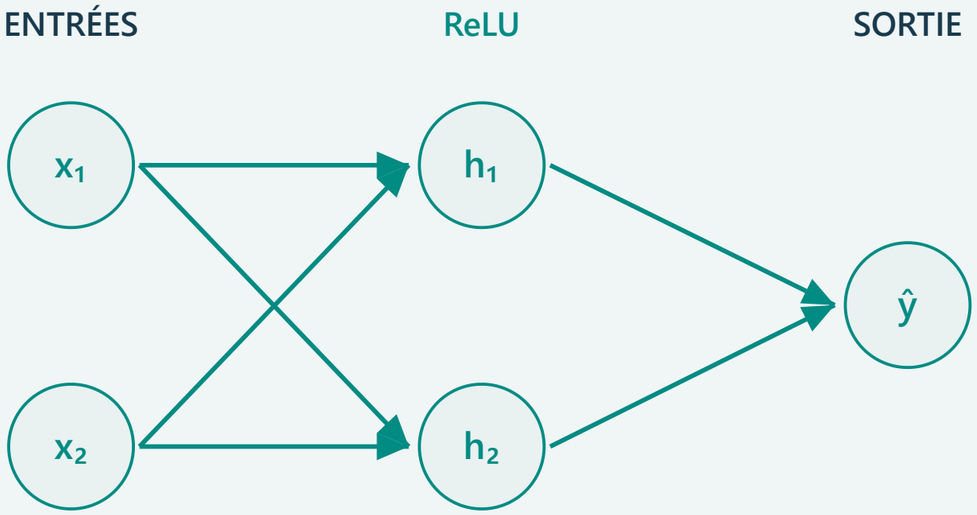
Un petit réseau complet, avant les grandes architectures.

Exemple pédagogique : 2 entrées → 2 neurones cachés → 1 sortie. Ce n'est pas le réseau du TD.

Couche : des coefficients multiplient les entrées, puis des biais sont ajoutés.

Activation ReLU : max(0, z) ; elle permet une relation non linéaire.

Sortie : une valeur prédite ; 9 paramètres et 6 multiplications-accumulations.



6 poids + 3 biais = 9 paramètres

2 × 2 + 2 × 1 = 6 MAC, biais exclus

Faire traverser des nombres au réseau.

Modifier une entrée change les activations et la prédiction. Les poids restent fixes.

> **Entrées :** $x_1 = 1$ et $x_2 = 2$ au départ ; les biais valent zéro.

> **Cachée :** $h_1 = \max(0, x_1 - x_2)$; $h_2 = \max(0, 0{,}5x_1 + 0{,}5x_2)$.

> **Sortie :** $\hat{y} = 2h_1 + h_2$; au départ, $\hat{y} = 1{,}5$.

INFÉRENCE · POIDS FIXES

x_1 1

x_2 2

h_1	h_2
0	1,5

$\hat{y} = 1{,}5$

h_1, h_2 : activations après ReLU. Les biais sont nuls ici ; aucun poids ne change.

Apprendre : mesurer l'erreur, puis corriger un poids.

Exemple limité à un poids de sortie w ; tous les autres paramètres restent fixes.

Cible c = 2 : la perte $L = (\hat{y} - c)^2$ vaut 0,25 au départ.

Gradient : $\partial L/\partial w = 2(\hat{y} - c)h_2 = -1{,}5$; il indique la pente.

Pas η = 0,1 : w passe de 1 à 1,15 ; recalculer ensuite la sortie et la perte.

UNE CORRECTION · UN SEUL POIDS

Pas η 0,1

Poids après correction	Nouvelle sortie ŷ
1,15	1,725

Perte avant	Perte après
0,25	0,075625

La perte diminue sur cet exemple.

$h_2 = 1{,}5$ et cible c = 2 sont fixes. Une correction n'est pas un entraînement complet.

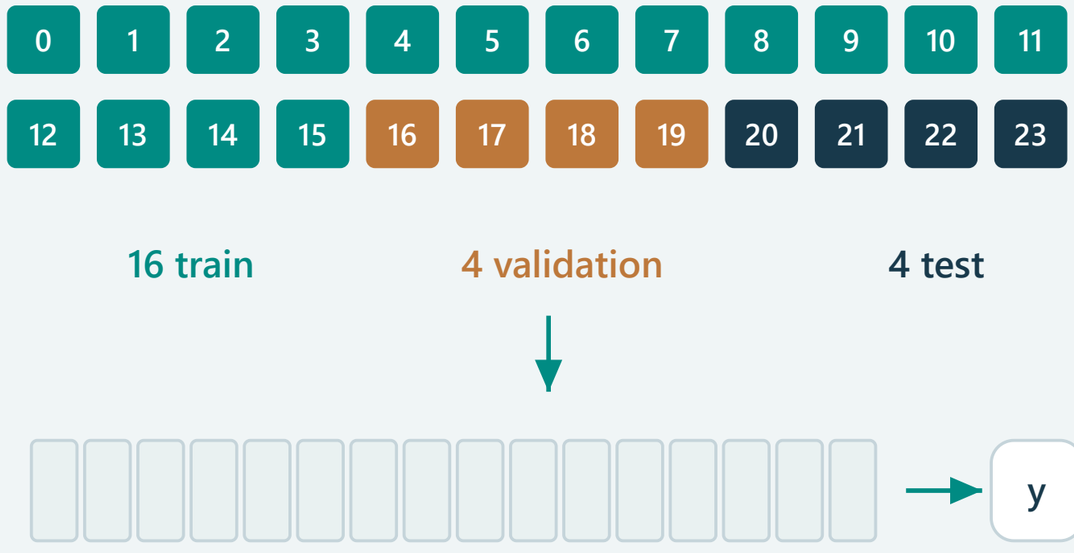
Des données pour apprendre. D’autres pour vérifier.

Une bonne courbe d’apprentissage ne suffit pas à démontrer une bonne prédiction.

> **Apprentissage :** ajuster poids et statistiques de normalisation.

> **Validation :** choisir les réglages, sans toucher au test réservé.

> **Test :** utiliser des séquences indépendantes ; comparer aussi à une méthode simple.



Une fenêtre reste dans une seule séquence.

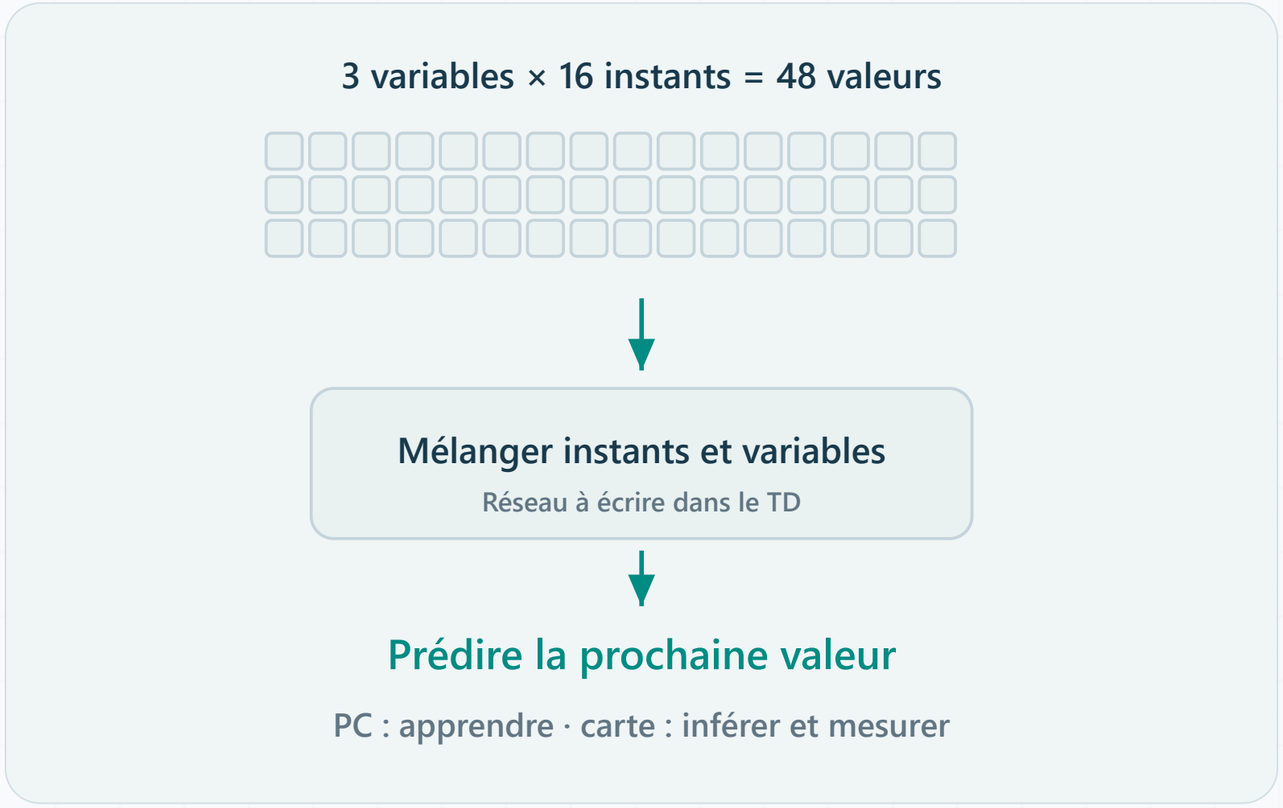
Du petit exemple au réseau que vous construirez.

La lampe explique le besoin ; les signaux simulés évitent de dépendre d'un capteur pour le TD.

Entrée du TD : 3 signaux observés pendant 16 instants.

Modèle : petit Mixer inspiré de TSMixer ; mélanger instants et variables.

Sortie : une prochaine valeur ; entraîner sur PC, puis mesurer l'inférence sur carte.



Des familles différentes, pour des structures différentes.

Choisir selon les données et le budget ; les détails avancés restent en annexes.

Convolution : réutiliser un motif local, par exemple dans une image.

Récurrence : conserver un état pour traiter un flux.

Attention : calculer des relations dépendantes du contenu ; leur mémoire compte.



Combien ce réseau contient-il de paramètres ?

Deux entrées, deux neurones cachés, une sortie. Toutes les connexions sont présentes, avec un biais par neurone calculé.

?

Votre raisonnement avant la réponse.

Comptez les poids, puis les biais. La question porte uniquement sur les paramètres.

- **A** 6 paramètres
- **B** 9 paramètres
- **C** 12 paramètres

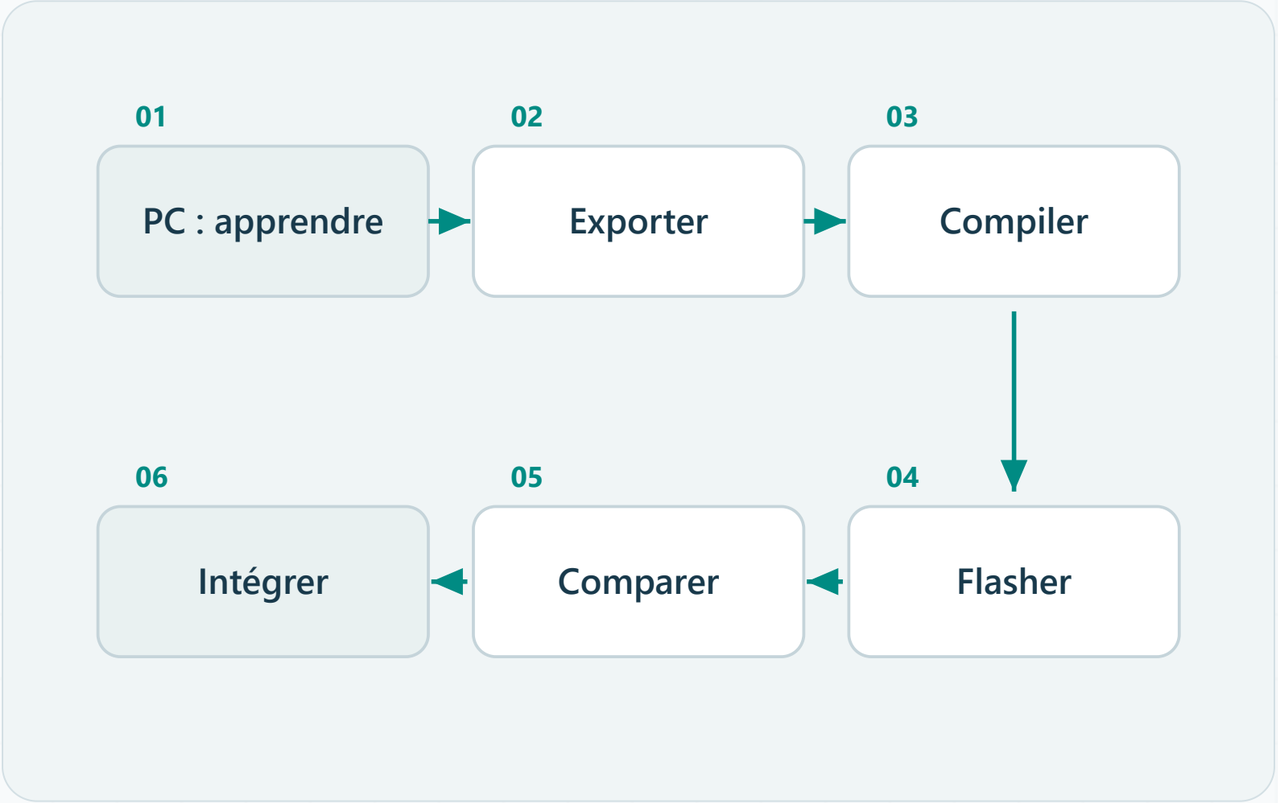
Du PC à la carte : qu'est-ce qui voyage ?

On entraîne sur PC. On porte le calcul et les poids pour prédire sans le PC.

PC : apprendre → figer les poids et la normalisation → exporter.

Carte : compiler ou installer → charger → exécuter l'inférence.

Vérifier : mêmes entrées, sorties comparées avec une tolérance expliquée.



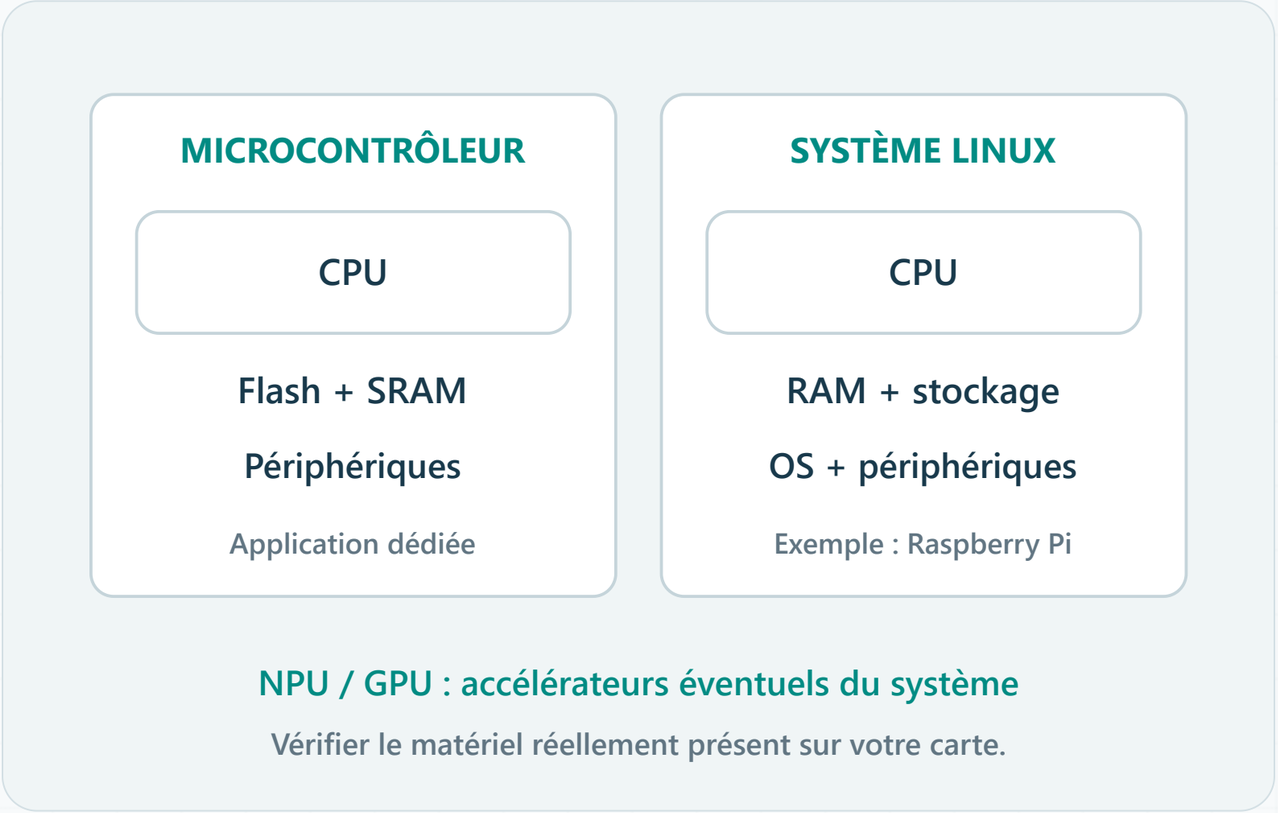
Une plateforme contient des unités de calcul.

Un MCU ou un système Linux peut réunir CPU, mémoires et accélérateurs.

CPU : exécute le programme général ; le MCU en contient un.

MCU : petit système intégré avec Flash, SRAM et périphériques.

Pi / SoC : autre budget et système ; GPU ou NPU sont des accélérateurs éventuels.



Le fichier ne raconte pas toute la mémoire.

Distinguer ce qui est conservé hors tension de ce qui doit tenir pendant l'inférence.

Flash / stockage : programme, poids et constantes, selon leur placement.

RAM : entrées, activations, buffers temporaires et pile.

Pic : additionner ce qui existe au même moment ; vérifier aussi l'application entière.

EXEMPLE MCU · POIDS CONSERVÉS EN FLASH

Poids bruts	Travail SRAM
400 Ko	**120 Ko**
100 000 paramètres	activations + buffers + pile



Poids FP32 | Poids INT8 seuls

Budgets fictifs distincts. Échelles et métadonnées exclues du payload ; SRAM inchangée dans cet exemple.

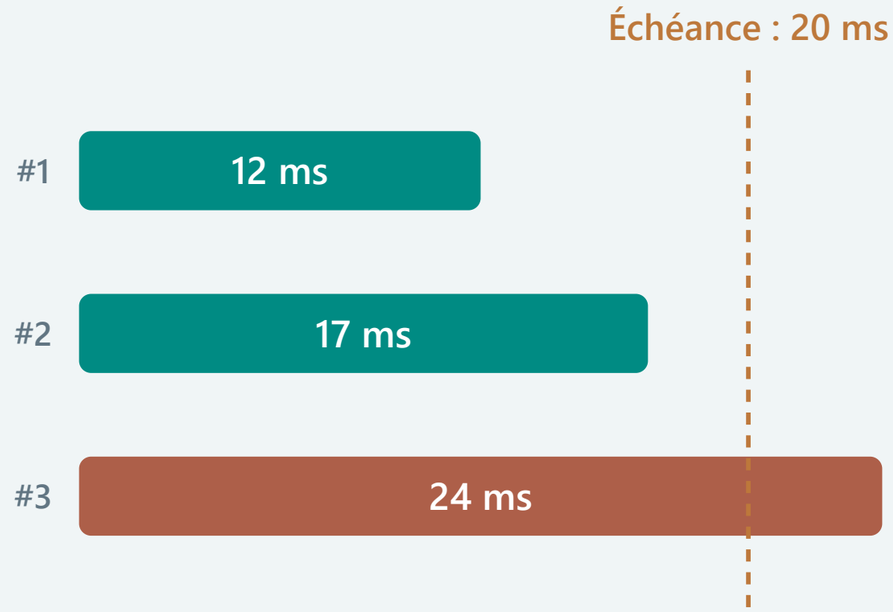
Une prédiction correcte peut arriver trop tard.

Le délai utile couvre acquisition, préparation, calcul et action.

Latence : temps pour une entrée ; débit : sorties par seconde.

Gigue : variation des durées ; mesurer la dispersion et les retards.

Échéance : définir quoi faire quand une réponse arrive trop tard.



Durées pédagogiques · le troisième appel est trop tardif.

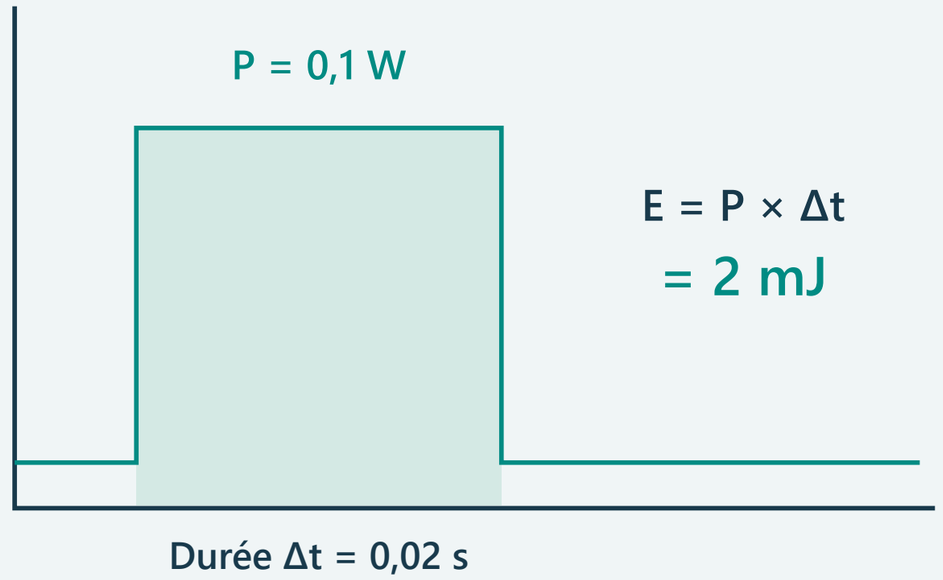
Plus rapide ne veut pas dire automatiquement plus sobre.

Puissance en watts, énergie en joules : on doit préciser ce que l'on mesure.

> **Énergie :** E = P × Δt si la puissance P est constante pendant Δt.

> **Exemple :** 0,1 W pendant 0,02 s = 0,002 J, soit 2 mJ.

> **Périmètre :** compter calcul, capteur, veille, radio et pertes selon le service.



Exemple à puissance constante · pas une mesure de carte

Quel est le pic de mémoire RAM annoncé ?

En RAM, simultanément : activations 60, temporaires 40, pile 20, entrées-sorties 16 Kio. Les 80 Kio de poids restent en Flash.

?

Votre raisonnement avant la réponse.

Additionnez uniquement ce qui occupe la RAM au même moment.

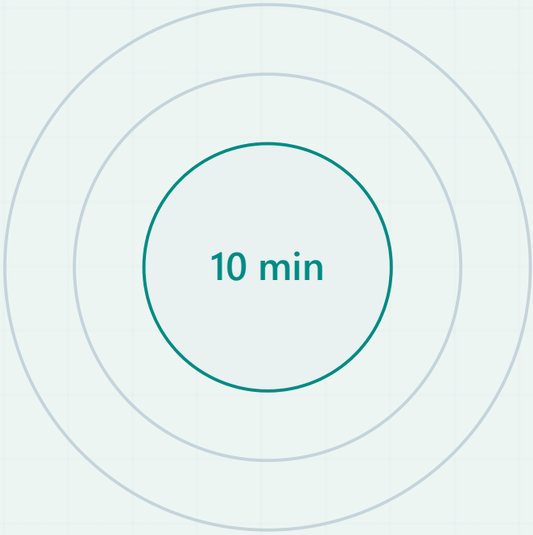
- **A** 120 Kio
- **B** 136 Kio
- **C** 216 Kio

Pause · 10 minutes.

Au retour : observer un goulot, proposer un changement et mesurer.

> **À retenir :** un service, un modèle, une chaîne complète.

> **À compter :** qualité, mémoire, temps et énergie.



Le modèle et son matériel forment un système.

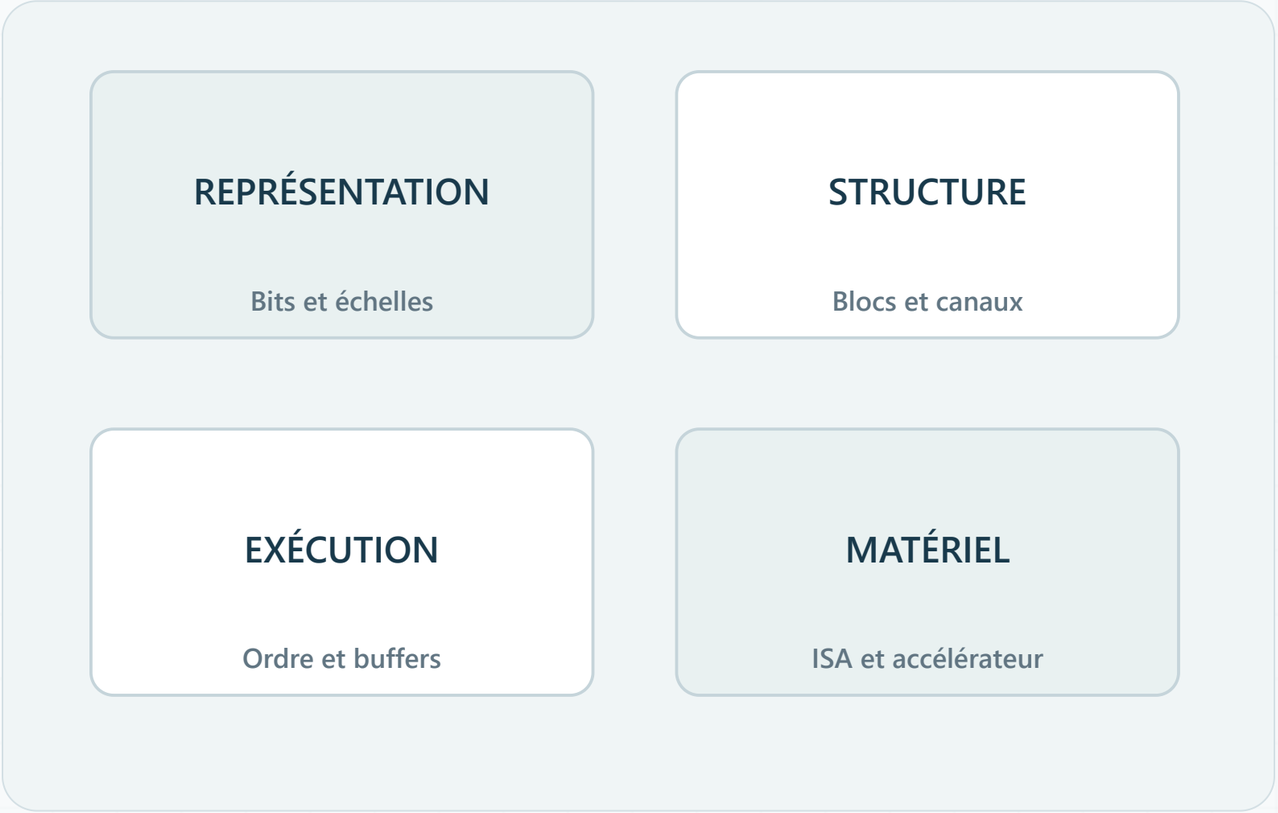
Mesurer d’abord. Choisir le levier ensuite.

Une optimisation répond à un coût observé et à une hypothèse vérifiable.

> **Poids trop volumineux :** tester une représentation plus petite.

> **Trop d’échanges mémoire :** revoir ordre, réemploi ou buffers.

> **Qualité insuffisante :** revoir données et modèle avant d’accélérer une erreur.



Levier 1 · Garder moins de chiffres.

La quantification arrondit sur une grille. Il faut vérifier l'effet sur la sortie.

Échelle s : distance entre deux valeurs représentables.

Recette : arrondir x/s, borner l'entier q, puis reconstruire $\hat{x} = s \times q$.

Limite : des poids plus petits ne garantissent ni qualité conservée ni exécution entière rapide.

ARRONDI SYMÉTRIQUE · EXEMPLE SUR [−1, 1]



12 valeurs illustratives

Précision des codes **4 bits**

Pas s : **0,1429**

Erreur max illustrée : **0,07**

Cercles : valeurs initiales ; carrés : valeurs reconstruites. Pas un test de qualité de réseau.

Levier 2 · Déplacer moins de données inutiles.

Réutiliser une donnée proche du processeur peut éviter des transferts.

Ordre : parcourir les tableaux dans un ordre favorable.

Réemploi : garder une valeur utile plutôt que la relire.

Vérification : même fonction, tolérance numérique et mémoire temporaire comptée.

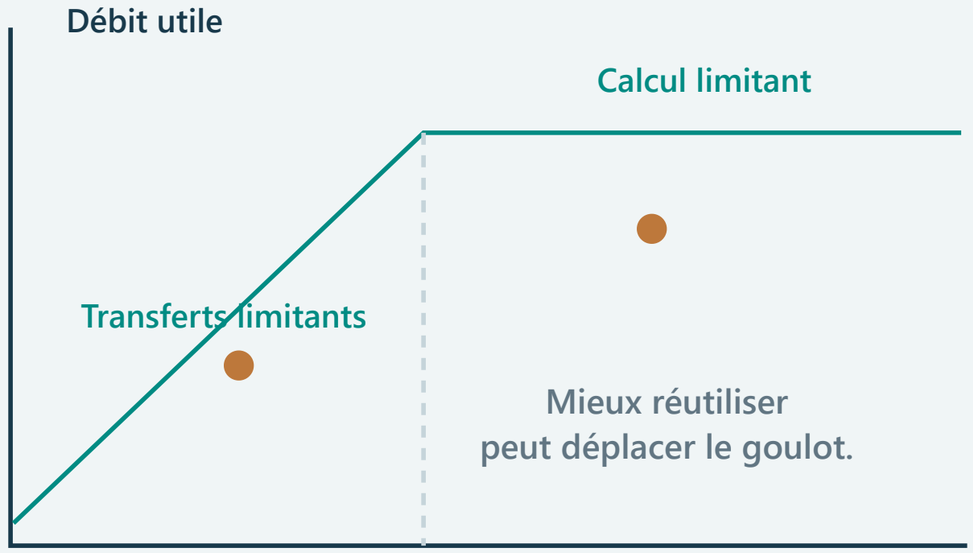


Schéma conceptuel · axes non calibrés

Micro-expérience : même somme, deux écritures.

Mesure réelle dans ce navigateur. Aucun résultat Raspberry Pi ou MCU n'est simulé.

> **Référence :** additionner un tableau avec une boucle.

> **Variante :** utiliser reduce ; vérifier l'égalité avant le chronométrage.

> **Protocole :** échauffement, 7 paires alternées, médiane, dispersion et CSV brut.

NAVIGATEUR ACTUEL · SOMME DE 4 096 VALEURS

Pas encore mesuré. Aucune durée préremplie.

Par somme	Médiane	Min–max
Boucle	—	—
reduce	—	—

Résultat vérifié avant et après. Données : Float64Array, 32 Kio ; ceci n'est pas un pic RAM système.

Les mesures varient avec le navigateur et sa charge. Aucune énergie ni durée sur carte n'est déduite.

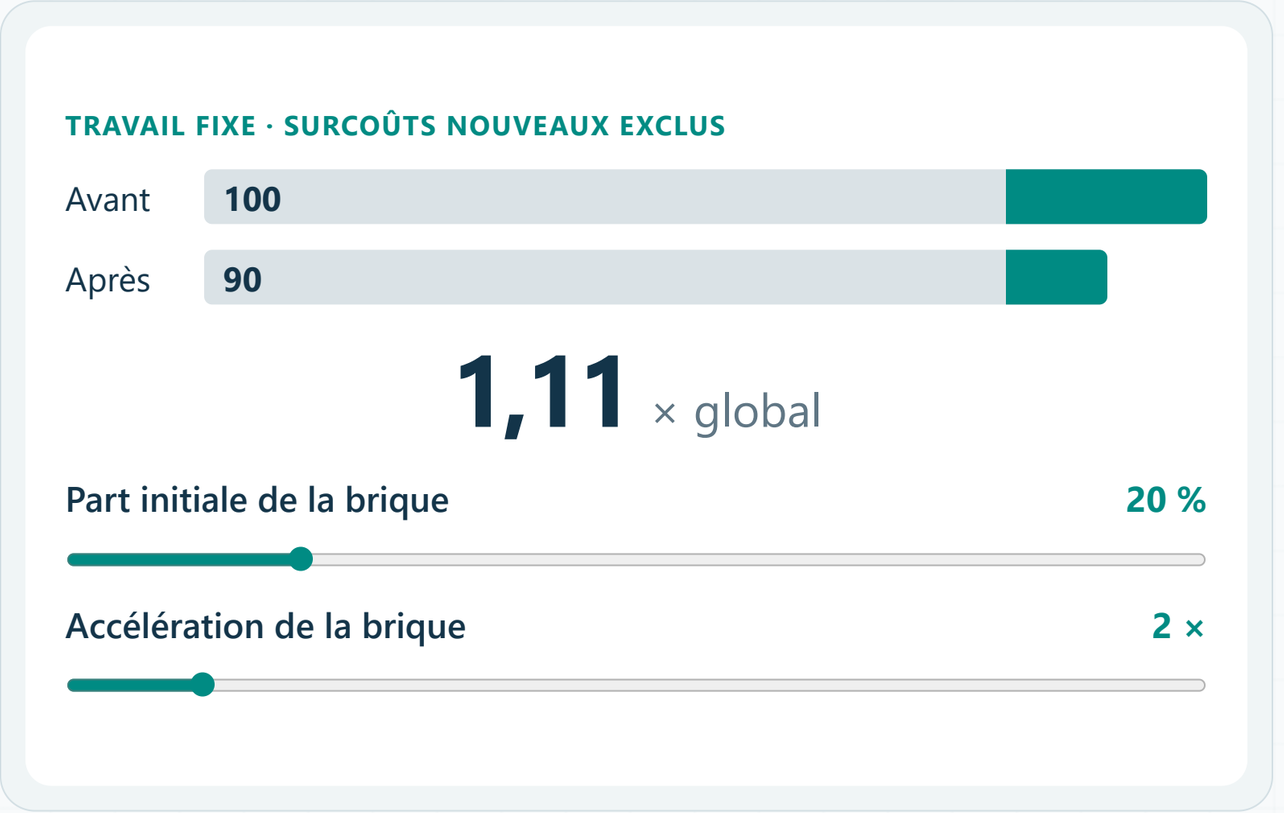
Un gain de brique n’est pas le gain du système.

La partie inchangée limite l’accélération globale : loi d’Amdahl.

> **p :** part du temps initial dans la brique ; s : accélération de cette brique.

> **Gain :** 1 / [(1 − p) + p/s], si aucun autre coût ne change.

> **Exemple :** p = 20 % et s = 2 donnent 1,11× ; 10 % de temps économisé.



Quel sera le nouveau délai total ?

Avant : 100 ms, dont 20 ms dans une brique. Cette brique devient deux fois plus rapide. Tout le reste est inchangé.

?

Votre raisonnement avant la réponse.

Séparez le temps de la brique et celui du reste, puis additionnez.

- **A** 90 ms
- **B** 50 ms
- **C** 80 ms

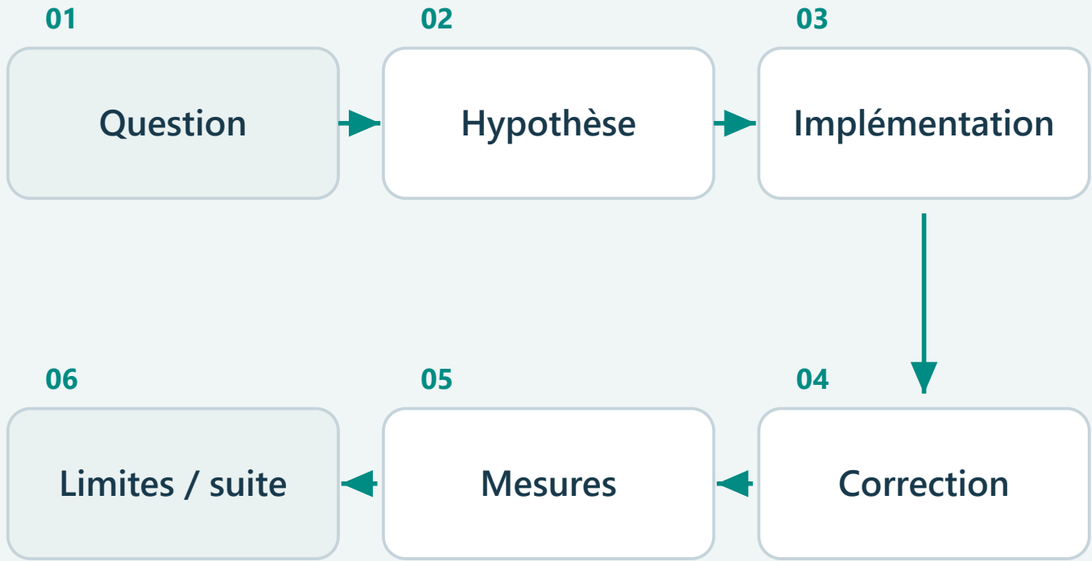
Une petite recherche : quand l'idée fonctionne-t-elle ?

Question du TD : quelle organisation d'une projection réduit son coût sur notre cible ?

Hypothèse : nommer le mécanisme attendu, pas seulement « aller plus vite ».

Comparaison : mêmes poids, données, compilation et sorties vérifiées.

Limite : mesurer aussi le réseau entier ; chercher quand le gain disparaît.



La conclusion peut être négative.

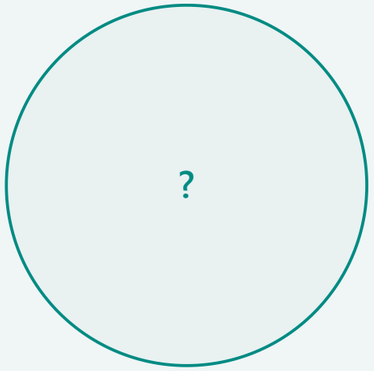
Prévoir les entrées difficiles et garder la maîtrise.

Une faible erreur moyenne ne suffit pas à rendre une action acceptable.

Tester : bruit, données absentes, valeurs hors plage et redémarrage.

Réagir : signaler, limiter ou refuser ; prévoir le mode de repli.

Protéger : contrôler les accès et les mises à jour ; le local n'est pas automatiquement sûr.



Argumenter avant de choisir.

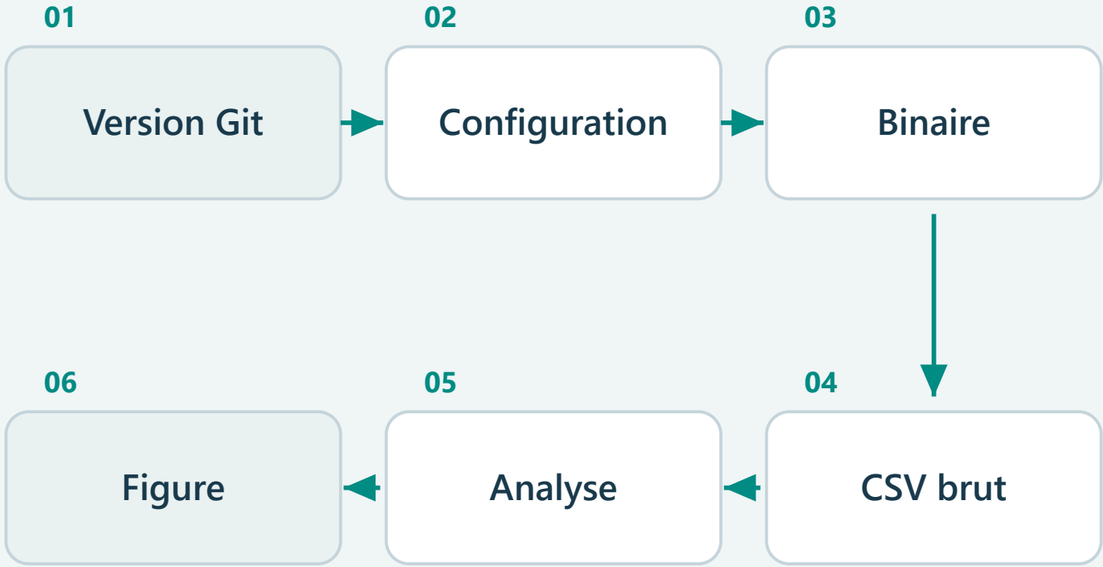
Un résultat doit retrouver son code et ses conditions.

commit → configuration → binaire → mesures brutes → figure → conclusion.

Versionner : étapes nommées, paramètres, outil de compilation et contributions.

Automatiser : tests de fonctions, contrôle des gradients et construction du programme.

Tracer : campagnes répétées, médiane, dispersion et fichiers bruts sur carte.



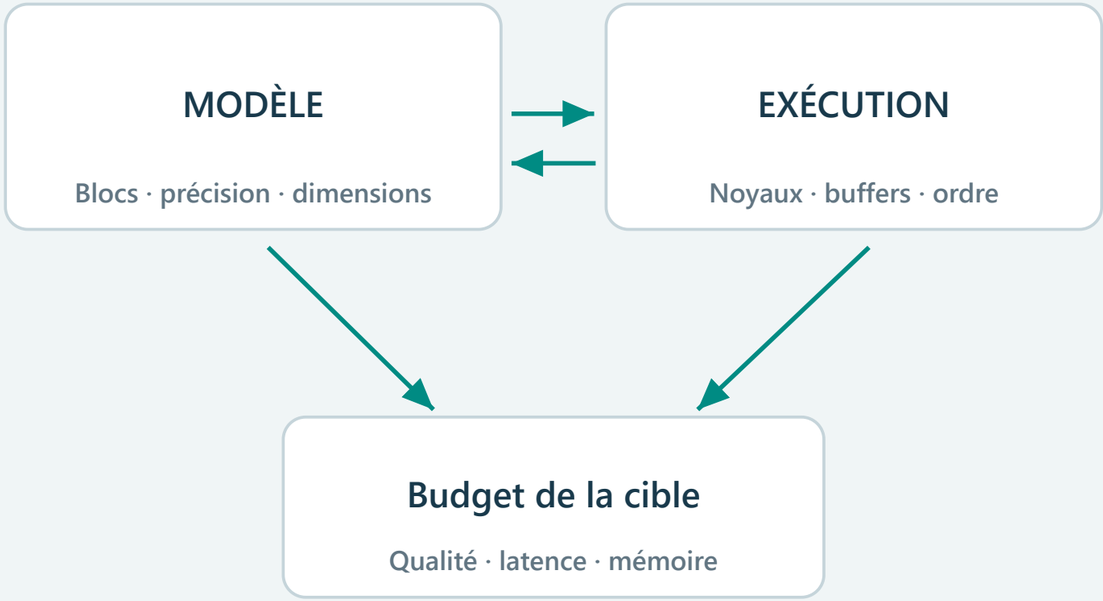
Lire un article comme un ingénieur.

MCUNet : faire tenir une tâche de vision en concevant ensemble réseau et moteur.

Problème : la mémoire de travail limite les réseaux sur microcontrôleur.

Réponse : adapter architecture et moteur, puis mesurer sur des cibles identifiées.

Votre lecture : tâche, matériel, référence, métriques, limites et expérience à reproduire.



Quel modèle respecte les quatre exigences ?

Tableau fictif. Exigences : qualité ≥ 96 %, RAM ≤ 100 Kio, p95 observé ≤ 20 ms et énergie ≤ 3 mJ par décision.

Valeurs fictives · périmètre commun

Modèle	Qualité	Kio	ms[1]	mJ
A	97 %	120	12	2,4
B	96 %	96	18	2,8
C	95 %	72	14	1,8

[1] p95 sur 500 essais fictifs. Énergie : acquisition + calcul + action, hors veille.

Vérifiez les quatre colonnes. Une exigence manquée élimine un candidat.

- **A** Le modèle A
- **B** Le modèle B
- **C** Le modèle C

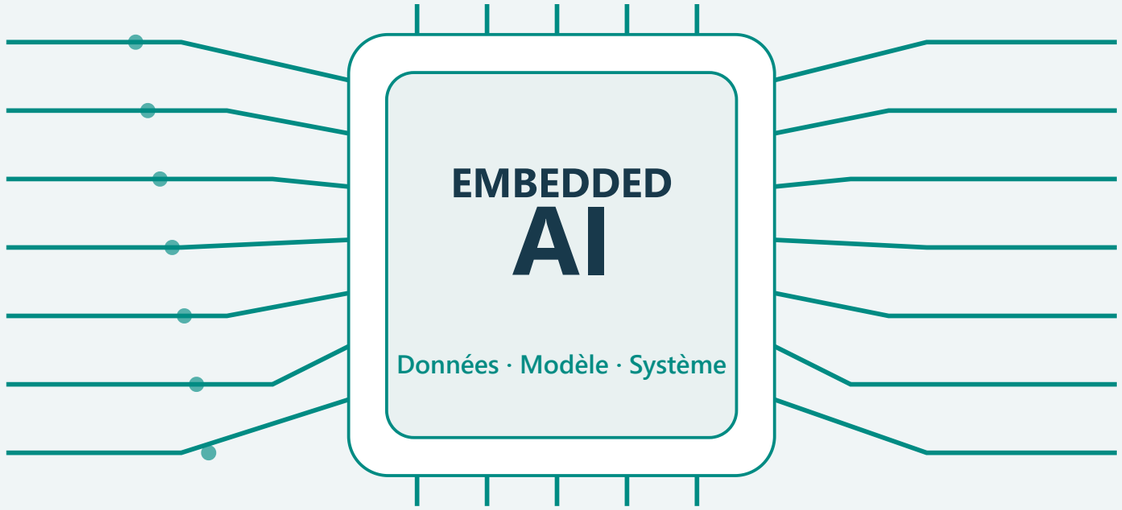
Ce que vous devez maintenant pouvoir démontrer.

Au TD, transformer cette compréhension en programme, en mesures et en explication.

Expliquer : une prédiction, un apprentissage et le trajet du PC à la carte.

Compter : paramètres, MAC, Flash, pic RAM et délai ; mesurer l'énergie si équipé.

Défendre : une optimisation vérifiée, ses résultats et ses limites.



Le service réel fixe le budget.

La convolution réutilise un petit motif.

Une convolution séparée découpe le filtrage spatial et le mélange des canaux.

Classique : chaque sortie combine espace et canaux.

Depthwise + 1×1 : filtrer chaque canal, puis les mélanger.

À vérifier : moins de MAC peut encore produire plus de transferts.



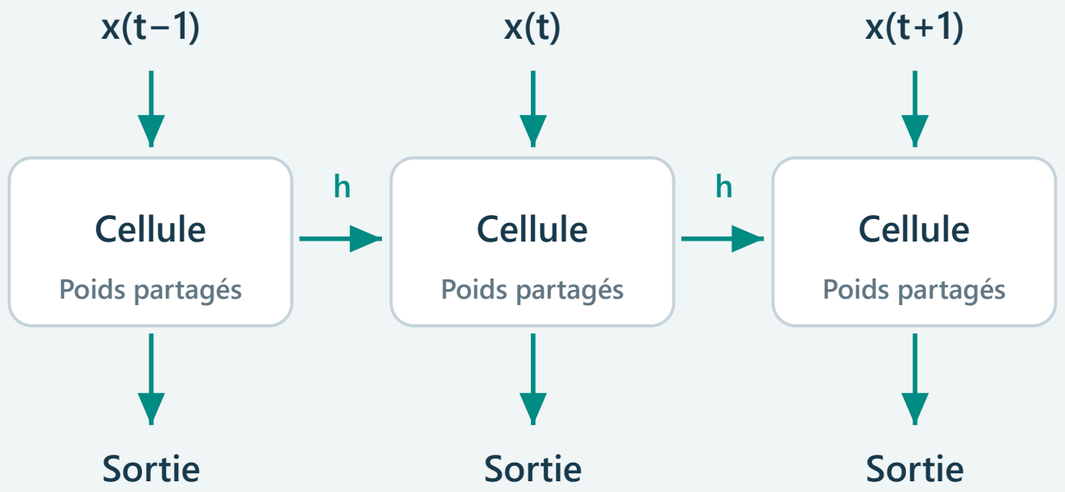
Un état mémorise une partie du passé.

Un réseau récurrent met à jour un petit état à chaque échantillon.

État h : résumé réutilisé au pas suivant.

Intérêt : traiter un flux sans garder tout son historique.

Limite : dépendance séquentielle, dérive et temps de mise en régime.



État transmis ≠ historique complet stocké

L’attention choisit quelles relations regarder.

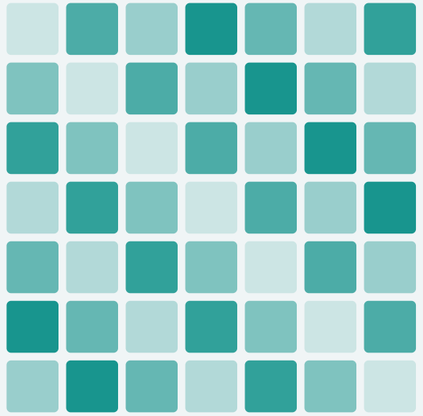
Au lieu d’un mélange fixe, les relations dépendent du contenu de l’entrée.

Q, K, V : requêtes, clés et valeurs calculées par projections.

Comparer : associer chaque position à d’autres positions.

Coût : une matrice naïve de scores grandit comme le carré de la longueur.

Scores entre positions



→

Normaliser puis mélanger

$\text{softmax}(QK^{T} / \sqrt{d})\, V$

^{T} : transpose · d : dimension d’une tête

Doubler la longueur → 4× plus de scores naïfs

Carte d’attention illustrative ; aucune sortie de modèle mesurée

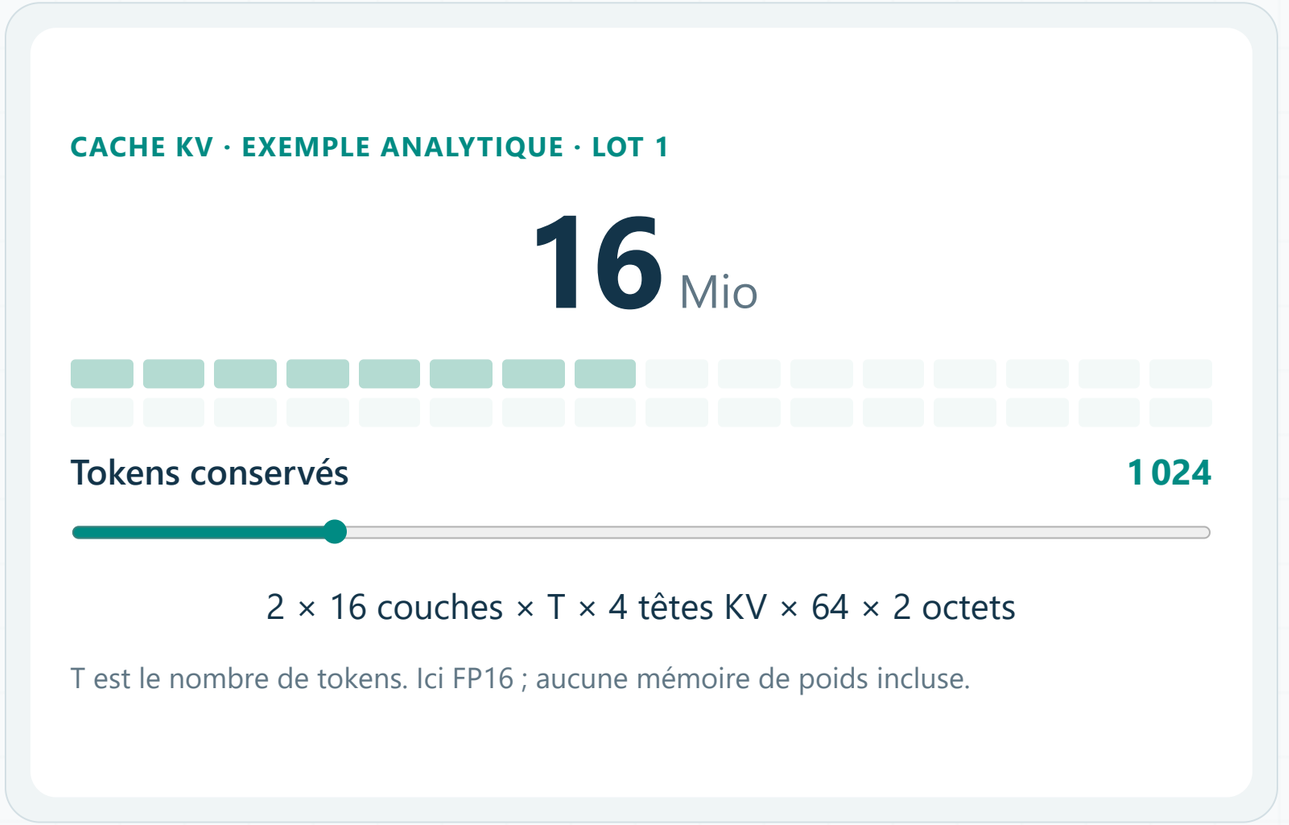
Le contexte consomme aussi de la mémoire.

Un cache KV évite de recalculer d'anciennes clés et valeurs lors de la génération.

Préremplissage : traiter la consigne initiale.

Décodage : produire ensuite un token à la fois.

Conséquence : réduire les poids ne réduit pas automatiquement le cache.



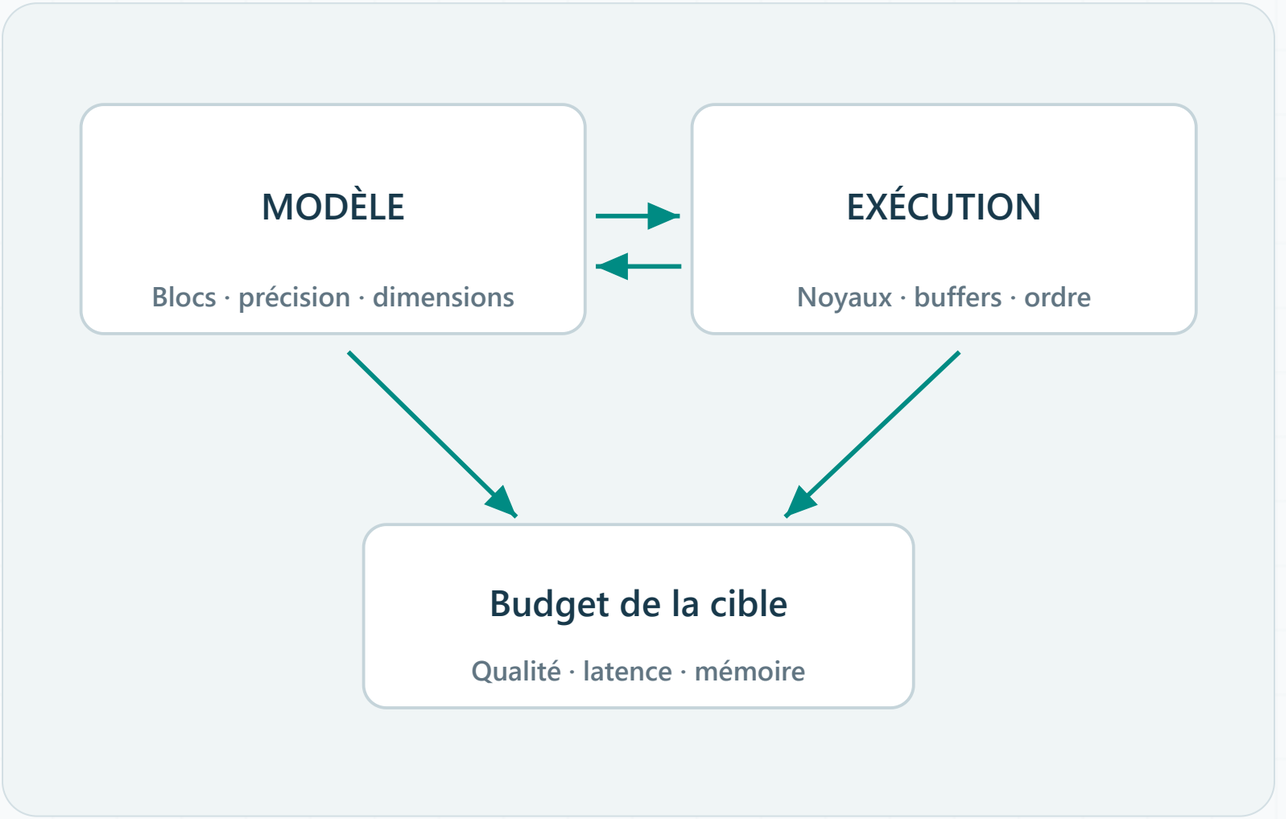
Deux recherches : déplacer la contrainte au bon endroit.

Architecture et exécution se répondent ; une seule métrique masque le compromis.

MCUNetV2 · 2021 : calculer par morceaux pour réduire le pic mémoire.

MobileNetV4 · 2024 : concevoir des blocs efficaces pour plusieurs cibles mobiles.

Question ouverte : le gain reste-t-il vrai sur notre cible et notre tâche ?



Calibrer, entraîner, puis comparer le vrai calcul.

La meilleure précision n'est pas forcément uniforme dans tout le réseau.

PTQ : quantifier après entraînement, avec calibration si nécessaire.

QAT : simuler les effets de quantification pendant l'apprentissage.

AWQ · 2024 : protéger les canaux sensibles à partir des activations.



Préserver ce qui change le plus la sortie

Puis vérifier qualité et noyau réellement exécuté.

Supprimer, simplifier, ou apprendre à imiter.

Compresser un fichier ne suffit pas à accélérer son exécution.

Élagage : retirer poids ou canaux ; exploiter réellement la structure obtenue.

Distillation : entraîner un petit élève à reproduire un enseignant.

Choix d'architecture : chercher un compromis compatible avec la cible.



Le noyau doit exploiter la structure restante.

Distillation : enseignant → petit élève à entraîner

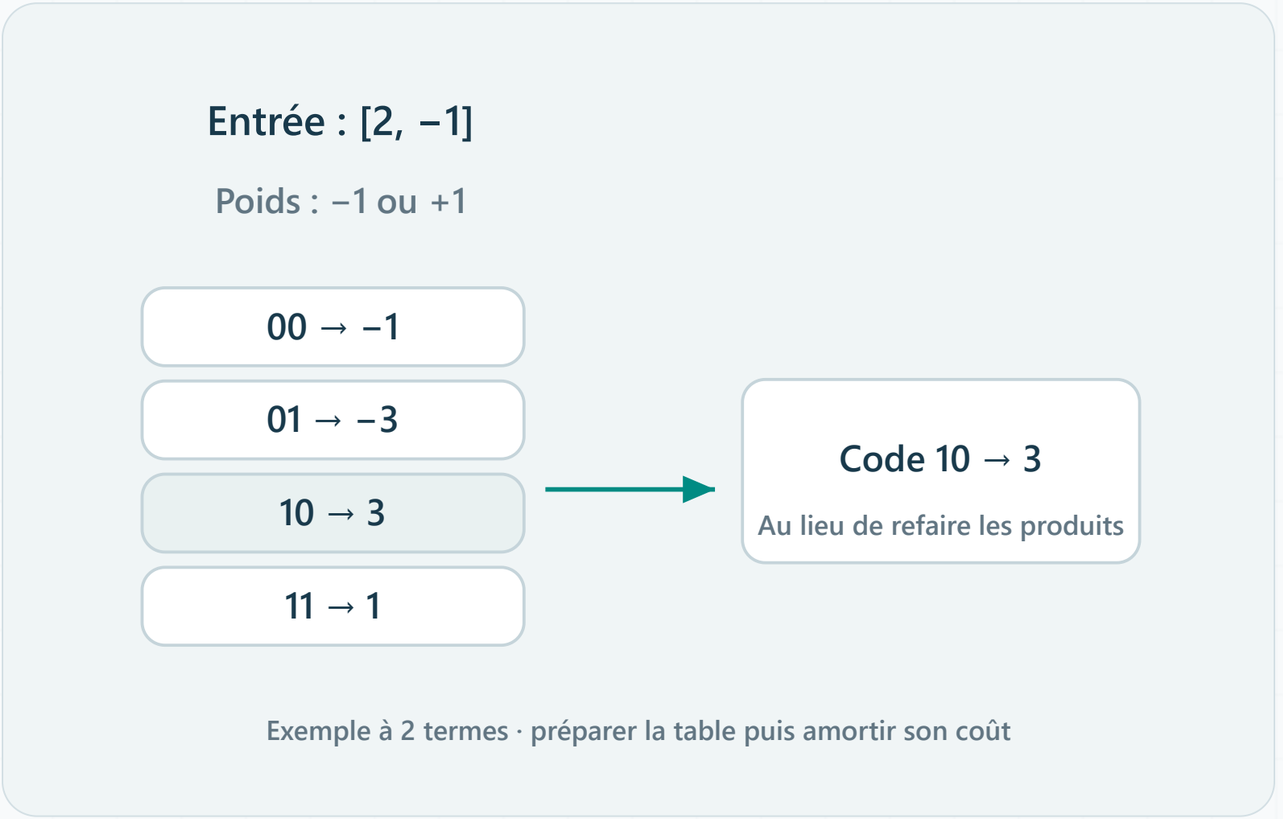
T-MAC · Remplacer un calcul par une consultation de table.

Jianyu Wei et al. · EuroSys 2025 · Une idée système pour produits bas bits sur CPU.

> **Problème :** les poids bas bits peuvent coûter cher à déquantifier.

> **Idée :** préparer des combinaisons, puis les retrouver par indices.

> **Limite :** la table et ses accès doivent être moins coûteux que le calcul évité.



« Petit modèle de langage » ne signifie pas « modèle pour MCU ».

Le mot petit dépend du point de comparaison.

> **1,7 milliard de paramètres à 4 bits :** environ 850 Mo de poids bruts.

> **À ajouter :** métadonnées, activations, cache KV et moteur.

> **À décider :** besoin réel de génération, ou classification spécialisée ?



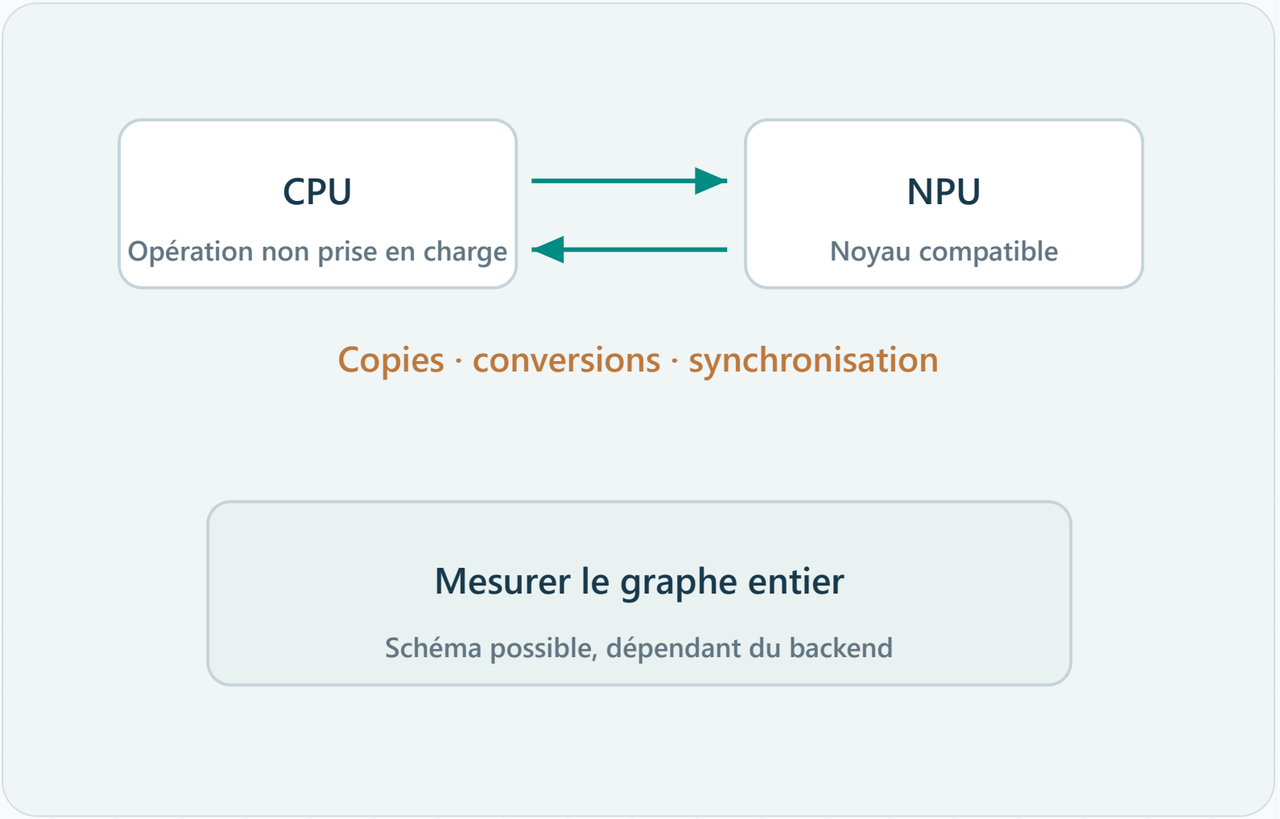
Veille 2025 · Le NPU ne gagne que si le graphe lui convient.

T-MAN · Wei, Li et al. · Prépublication : lire ce qui est accéléré, et ce qui ne l'est pas.

> **Problème :** des opérations périphériques limitent un accélérateur spécialisé.

> **Piste :** adapter tables, tuilage et pipeline.

> **À contrôler :** préremplissage, décodage, transferts et comparaison de départ.



Veille 2026 · Deux voies, deux niveaux d'intervention.

Des pistes actuelles à examiner ; aucune promesse universelle de performance.

FastGRNN · Kizilates : très petite récurrence, tables et validation sur MCU.

RISC-V · Kumar et al. : instructions et compilation pour Transformers compacts.

Regard critique : mise en régime, graines, mémoire, aire et protocole.

JUIN 2026

FastGRNN sur MCU

Mise en régime à examiner

Emre Can Kizilates

OCTOBRE 2026

Transformer sur RISC-V

Instructions + compilation
Aire et énergie à compter

Ajay Kumar M et al.

Deux prépublications : hypothèses et périmètres à vérifier.

Sources et limites de lecture · 1/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R01 ALVINN: An Autonomous Land Vehicle In a Neural Network ↗
Dean Pomerleau · 1989 · Repère historique · publication CMU / actes NeurIPS
Relier perception apprise et action à bord d'un véhicule.

R02 MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications ↗
Andrew G. Howard et al. · 2017 · Article fondateur · arXiv 1704.04861
Séparer convolution spatiale et mélange des canaux pour réduire le calcul.

R03 Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ↗
Benoit Jacob et al. · 2018 · CVPR 2018 · prépublication 2017
Co-concevoir quantification, apprentissage et noyaux entiers.

R04 MCUNet: Tiny Deep Learning on IoT Devices ↗
Ji Lin, Wei-Ming Chen et al. · 2020 · NeurIPS 2020
Chercher ensemble l'architecture du réseau et son moteur d'exécution.

Sources et limites de lecture · 2/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R05 MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning ↗
Ji Lin, Wei-Ming Chen, Han Cai et al. · 2021 · NeurIPS 2021
Traiter des morceaux de carte de caractéristiques pour réduire le pic mémoire.

R06 On-Device Training Under 256KB Memory ↗
Ji Lin, Ligeng Zhu et al. · 2022 · NeurIPS 2022
Rendre possible une adaptation locale par mises à jour sélectives et co-conception.

R07 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness ↗
Tri Dao, Daniel Y. Fu et al. · 2022 · NeurIPS 2022
Réorganiser le calcul d'attention pour réduire les échanges mémoire sur GPU.

R08 TSMixer: An All-MLP Architecture for Time Series Forecasting ↗
Si-An Chen et al. · 2023 · Transactions on Machine Learning Research, 2023
Mélanger les instants puis les variables avec des perceptrons multicouches.

Sources et limites de lecture · 3/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R09 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration ↗
Ji Lin, Jiaming Tang et al. · 2024 · MLSys 2024 · prépublication 2023
Utiliser des statistiques d'activation pour mieux préserver les canaux sensibles.

R10 MobileNetV4 — Universal Models for the Mobile Ecosystem ↗
Danfeng Qin et al. · 2024 · ECCV 2024
Concevoir des blocs et une recherche d'architecture adaptés à plusieurs matériels mobiles.

R11 T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge ↗
Jianyu Wei, Shijie Cao et al. · 2025 · EuroSys 2025 · prépublication 2024
Remplacer certaines opérations de produits mixtes par des tables de recherche adaptées.

R12 SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model ↗
Loubna Ben Allal et al. · 2025 · Prépublication arXiv 2502.02737
La qualité et le mélange des données comptent aussi pour les petits modèles de langage.

Sources et limites de lecture · 4/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R13 T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup ↗
Jianyu Wei, Qingtao Li et al. · 2025 · Prépublication arXiv · 14 novembre 2025
Adapter tables, tuilage et pipeline aux opérations réellement prises en charge par un NPU.

R14 From Compression to Deployment: Real-Time and Energy-Efficient FastGRNN on Ultra-Constrained Microcontrollers ↗
Emre Can Kizilates · 2026 · Prépublication arXiv · 15 juin 2026
Étudier ensemble compression, activation par table, état récurrent et déploiement physique.

R15 A Framework for Accelerating Transformer Inference on RISC-V for Edge AI ↗
Ajay Kumar M et al. · 2026 · Prépublication arXiv · 6 octobre 2026
Adapter instructions et compilation à des Transformers compacts, principalement encodeurs.

R16 Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations ↗
Apostol Vassilev, Alina Oprea et al. · 2025 · Rapport technique NIST AI 100-2e2025
Distinguer attaques sur données, entrées, modèles et vie privée.

Sources et limites de lecture · 5/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R17 Tiny Machine Learning: Progress and Futures ↗
Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Song Han · 2023 / 2024 · IEEE Circuits and Systems Magazine 2023 · dépôt arXiv 2024
Vue d'ensemble de la co-conception et du passage de l'inférence à l'adaptation locale.

R18 MLPerf Tiny Benchmark ↗
Colby Banbury, Vijay Janapa Reddi et al. · 2021 · NeurIPS 2021 · Datasets and Benchmarks
Comparer qualité, latence et énergie avec un protocole reproductible.

R19 CMSIS-NN — Efficient neural network kernels for Arm Cortex-M ↗
Arm · Documentation consultée en 2026 · Dépôt officiel de noyaux optimisés
Les performances dépendent de noyaux adaptés à l'ISA et aux formats quantifiés.

R20 LiteRT for Microcontrollers ↗
Google AI Edge · Documentation consultée en 2026 · Documentation officielle du moteur MCU
Déployer un sous-ensemble d'opérateurs avec une allocation adaptée au microcontrôleur.

Sources et limites de lecture · 6/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R21 MLPerf Inference: Tiny ↗
MLCommons · Documentation consultée en 2026 · Méthodologie et résultats officiels
Lire conjointement qualité, scénario de mesure et configuration matérielle.

R22 Shakey the Robot ↗
SRI International · 1966–1972 · Histoire du projet · organisme concepteur
Faire percevoir un environnement et planifier des actions à un robot mobile.

R23 Edge AI ↗
NIST · Documentation consultée en 2026 · Programme de recherche officiel
Rapprocher l'inférence ou l'apprentissage des données et des usages.

Lexique · Matériel et mémoire

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

MCU

Microcontrôleur : processeur, mémoire et périphériques réunis pour une application embarquée.

CPU / GPU / NPU

Calcul général / calcul parallèle / accélération spécialisée de réseaux.

Flash

Mémoire non volatile : programme et constantes, selon le placement choisi.

SRAM / RAM

Mémoire de travail volatile. Compter les données simultanément vivantes.

SIMD

Une instruction traite plusieurs valeurs ; les formats et alignements comptent.

DMA

Transfert de données piloté par un contrôleur ; bande passante et synchronisation restent limitées.

ISA

Jeu d'instructions du processeur. Un noyau doit exploiter ce qui est réellement disponible.

Ko / Kio

Ko = 1000 octets ; Kio = 1024 octets. Mio = 1024^2 octets dans ce cours.

Lexique · Modèles et expérimentation

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

Paramètre / activation

Un paramètre est appris ; une activation dépend de l'entrée en cours de calcul.

MAC

Multiplier puis accumuler. Préciser la convention avant de comparer des opérations.

Perte / gradient

La perte mesure une erreur ; le gradient indique comment elle varie avec les paramètres.

PTQ / QAT

Quantification après entraînement / apprentissage tenant compte de la quantification.

Cache KV

Clés et valeurs conservées pour éviter certains recalculs pendant la génération.

Runtime / backend

Moteur qui exécute le graphe / implémentation qui choisit les noyaux de la cible.

Latence / débit

Temps d'une réponse / quantité de réponses par unité de temps. Ils ne se confondent pas.

CI / manifeste

Tests et construction automatisés / fiche qui relie version, conditions et fichiers de l'expérience.

Réponses raisonnées · 1/2

L’argument compte davantage que la lettre choisie.

Question 1 · réponse A · diapositive 6

A respecte les deux exigences. Avec B, l’audio quitte l’objet. Avec C, la commande dépend aussi du réseau.

Question 2 · réponse B · diapositive 13

La couche cachée a 4 poids et 2 biais ; la sortie a 2 poids et 1 biais. Total : 9 paramètres.

Question 3 · réponse B · diapositive 19

60 + 40 + 20 + 16 = 136 Kio. Les poids restent ici en Flash. Avec 128 Kio de RAM, il manque au moins 8 Kio.

Réponses raisonnées · 2/2

L’argument compte davantage que la lettre choisie.

Question 4 · réponse A · diapositive 26

Le reste prend toujours 80 ms. La brique passe de 20 à 10 ms : total 90 ms, soit 10 % de temps économisé.

Question 5 · réponse B · diapositive 31

B respecte les quatre seuils. A dépasse la RAM ; C n’atteint pas la qualité demandée. Ce choix reste à valider dans d’autres conditions.